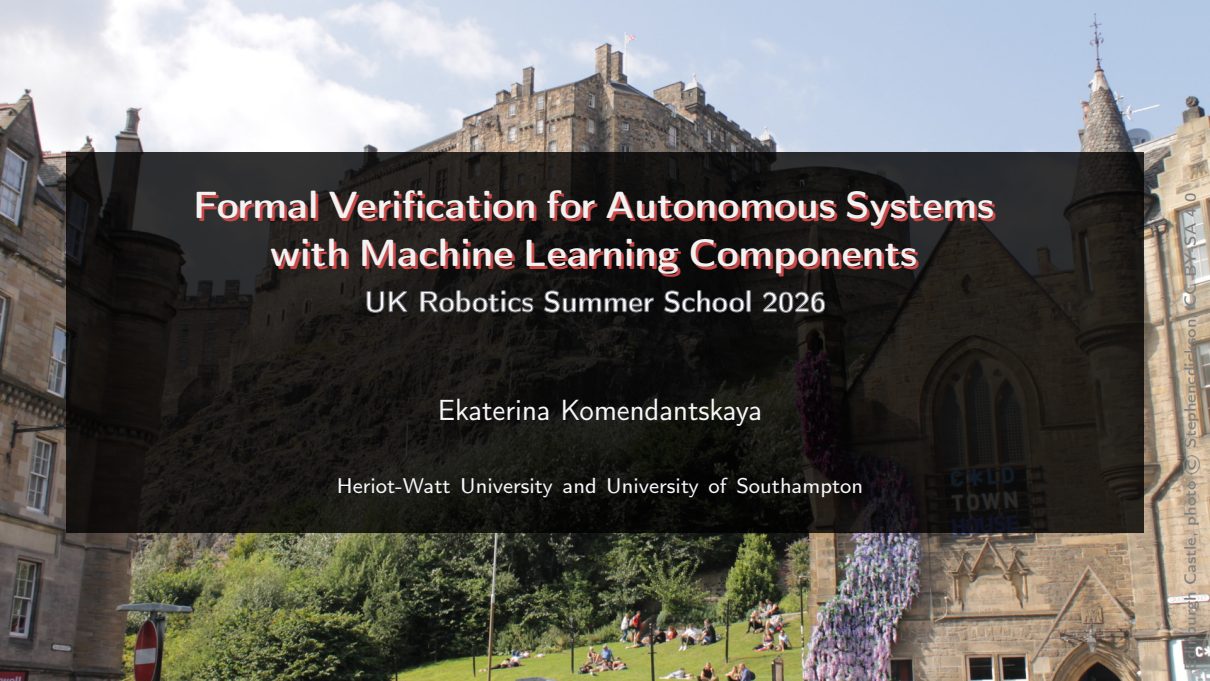


The background image is a composite of two photographs. The top half shows Edinburgh Castle, a large stone fortress with multiple towers and battlements, perched on a rocky hill under a blue sky with scattered white clouds. The bottom half shows a lush green park with many people sitting on the grass, surrounded by trees. On the right side, there is a stone building with Gothic architectural features, including a large arched window and a sign that reads 'CHILD TOWN HOUSE'. A purple flower garland hangs from the building.

Formal Verification for Autonomous Systems with Machine Learning Components

UK Robotics Summer School 2026

Ekaterina Komendantskaya

Heriot-Watt University and University of Southampton



Agenda for the Verification of AI in Robotics Day

- ▶ Introduction to Safe AI methods for Autonomous Systems.
- ▶ Verification of Neural networks with Vehicle.
- ▶ Training Neural networks to be safe, by Ekaterina Komendantskaya, U. Southampton & Heriot-Watt
- ▶ Verification of Medical Devices: an in-depth case study. By Alistair Sirman, Southampton University and Jessica Ciupa, Heriot-Watt University





Outline

ML in Robotics: What and Why we verify? Importance of Specification

Complexity of AI Verification: ML, Verification, Cyber-Physical Systems Perspectives

The "Hello World" example in ML Verification: the ACAS Xu benchmark

Specification-driven training



Pervasive Neural Networks...

Autonomous cars



Smart Homes



Robotics



Aircraft Controllers

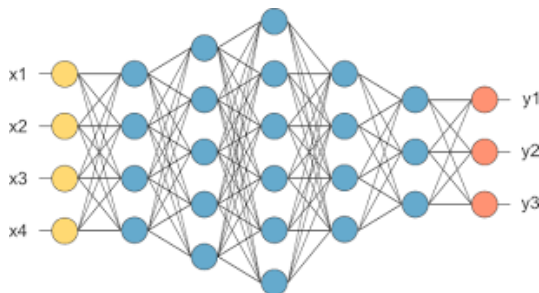


...and many more ...

... in need of verification: safety, security, robustness to adversarial attacks, ...



Neural network



A neural network is a function $f : \mathbb{R}^n \rightarrow \mathbb{R}^m$.
... We will also use the notation $f(x) = y$



Neural networks

Neural Networks are ideal for tasks where the semantics of the domain are poorly understood:

- ▶ they can learn to approximate complex functions.
- ▶ they are tolerant to noisy and incomplete data.

however...

- ▶ their solutions are not easily explainable.
- ▶ they are prone to a new range of bugs and safety and security problems.

Question: Can we verify properties of neural network-enhanced software?

Can we verify properties of neural network-enhanced software?





Can we verify properties of neural network-enhanced software?

1. Can we state the properties?



Property source 1: Universal properties

Properties that should hold of (almost) any neural network.



Example 1: Adversarial robustness as a PROBLEM

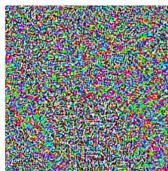
 Christian Szegedy, Wojciech Zaremba, Ilya Sutskever, Joan Bruna, Dumitru Erhan, Ian J. Goodfellow, Rob Fergus: Intriguing properties of neural networks. ICLR (Poster) 2014



"red"

97.6% confidence

+ .007 ×



noise

=



"green"

81.2% confidence



Example 1: Adversarial robustness as a PROBLEM

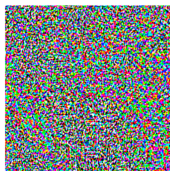
 Christian Szegedy, Wojciech Zaremba, Ilya Sutskever, Joan Bruna, Dumitru Erhan, Ian J. Goodfellow, Rob Fergus: Intriguing properties of neural networks. ICLR (Poster) 2014



"red"

97.6% confidence

+ .007 ×



noise

=



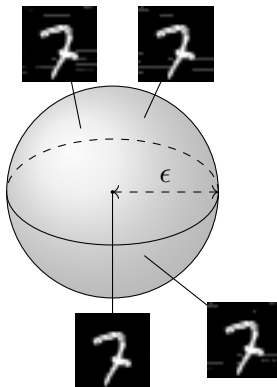
"green"

81.2% confidence

- ▶ the perturbations are imperceptible to human eye
- ▶ attacks transfer from one neural network to another
- ▶ affect any domain where neural networks are applied

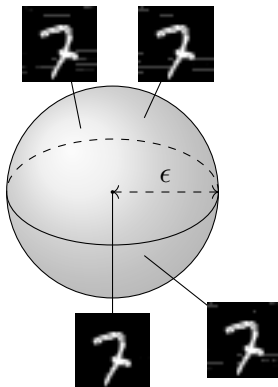


The “ ϵ -ball robustness” as a PROPERTY





The “ ϵ -ball robustness” as a PROPERTY



Given $\hat{x} \in S$, an ϵ -ball $\mathbb{B}(\hat{x}, \epsilon) = \{x \in \mathbb{R}^n : |\hat{x} - x| \leq \epsilon\}$

Classify all points in $\mathbb{B}(\hat{x}, \epsilon)$ “robustly”.



Example 1: Adversarial Robustness

Property: "small" changes in the input should produce "small" changes in the output.

Definition (Robustness)

Let $f \in \mathbb{R}^m \rightarrow \mathbb{R}^n$ be a neural network, $\hat{x} \in \mathbb{R}^m$ be an input and $\epsilon, \delta \in \mathbb{R}$ then the network is $\epsilon - \delta$ -robust around $\hat{x}$ iff:

$$\forall x \in \mathbb{R}^m : |x - \hat{x}| \leq \epsilon \Rightarrow |f(x) - f(\hat{x})| \leq \delta$$



PROBLEM vs PROPERTY

Note the difference between:

- ▶ a PROBLEM (perhaps vaguely defined, unsolved, with multiple or unknown causes) and
- ▶ a PROPERTY (exactly mathematically defined, can be implemented and formally proven, does not necessarily solve the root causes)

It is common in verification generally to have a gap between PROBLEM and PROPERTY.



PROBLEM vs PROPERTY

Note the difference between:

- ▶ a PROBLEM (perhaps vaguely defined, unsolved, with multiple or unknown causes) and
- ▶ a PROPERTY (exactly mathematically defined, can be implemented and formally proven, does not necessarily solve the root causes)

It is common in verification generally to have a gap between PROBLEM and PROPERTY.
Formal Verification is harder when the gap is larger.

In Robotics

Easy to say – this Robot may be unsafe (PROBLEM).
Hard to define the exact safety conditions (PROPERTIES).



Property source 2: Approximations of standard algorithms

Sometimes neural networks are used to approximate “standard” algorithms.



Property source 2: Approximations of standard algorithms

Sometimes neural networks are used to approximate “standard” algorithms.

Why? Mainly for performance reasons.



Property source 2: Approximations of standard algorithms

Sometimes neural networks are used to approximate “standard” algorithms.

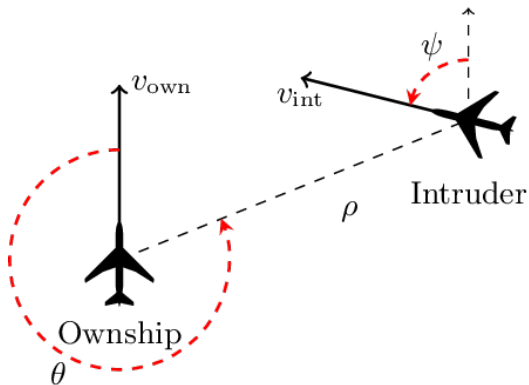
Why? Mainly for performance reasons.

We can derive desired properties of the network from those of the original algorithm.



Example 2: ACAS Xu

A collision avoidance system for unmanned autonomous aircraft.



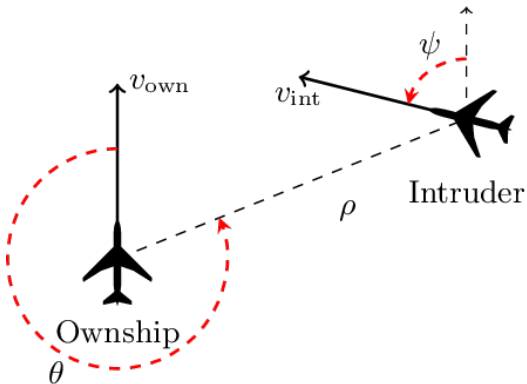


Example 2: ACAS Xu

A collision avoidance system for unmanned autonomous aircraft.

Inputs:

- ▶ Distance to intruder, ρ
- ▶ Angle to intruder, θ
- ▶ Intruder heading angle, ψ
- ▶ Speed, v_{own}
- ▶ Intruder speed, v_{int}





Example 2: ACAS Xu

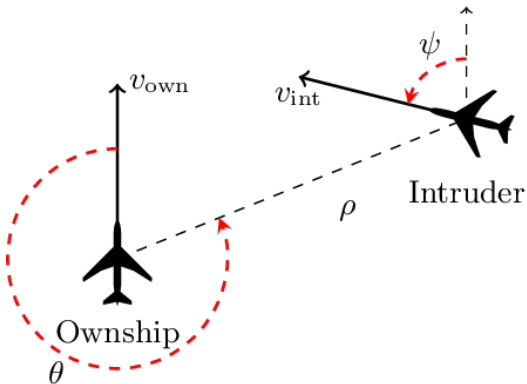
A collision avoidance system for unmanned autonomous aircraft.

Inputs:

- ▶ Distance to intruder, ρ
- ▶ Angle to intruder, θ
- ▶ Intruder heading angle, ψ
- ▶ Speed, v_{own}
- ▶ Intruder speed, v_{int}

Outputs:

- ▶ Clear of conflict
- ▶ Strong left
- ▶ Weak left
- ▶ Weak right
- ▶ Strong right





Example 2: ACAS Xu properties

Originally implemented as a 2Gb lookup table but was replaced with a neural network in order to improve size and latency requirements. 10 different domain-specific properties in total.

Definition (ACAS Xu: Property 3)

If the intruder is directly ahead and is moving towards the ownship, the score for COC will not be minimal.

(We will see the mathematical formulation of Property 3 shortly.)



Property source 3: Domain specific properties

Often there may be properties specific to the domain being modelled.

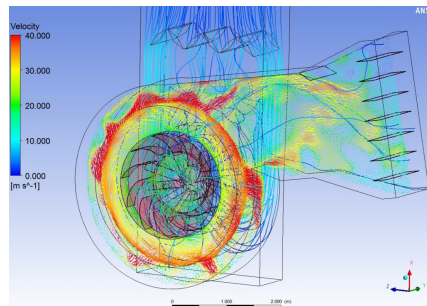


Example 3: fluid modelling

Consider trying to model a fluid in a tube.

Very difficult to do precisely, but we know:

- ▶ energy should be conserved.
- ▶ etc.





Properties: Summary

General rule:

Verification properties are properties that are pragmatically important (safety/security/fairness), but not guaranteed to occur in the sampled data.



Properties: Summary

General rule:

Verification properties are properties that are pragmatically important (safety/security/fairness), but not guaranteed to occur in the sampled data.

1. Robustness, smoothness, monotonicity
2. Properties coming from intended application domains, when neural nets approximate standard algorithms
3. Properties specific to the domain that the neural network models

Taking real industrial partners helps to understand the PROBLEMS and PROPERTIES better. [More on this later](#)



Outline

ML in Robotics: What and Why we verify? Importance of Specification

Complexity of AI Verification: ML, Verification, Cyber-Physical Systems Perspectives

The "Hello World" example in ML Verification: the ACAS Xu benchmark

Specification-driven training

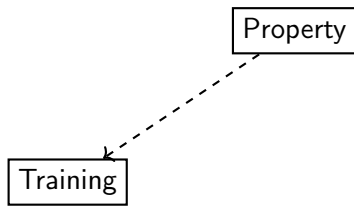


The lifecycle of neural network verification

Property



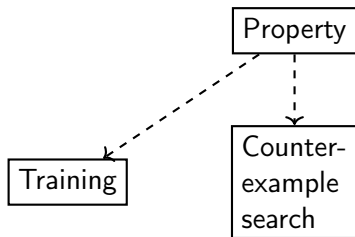
The lifecycle of neural network verification



DL2
ACT
etc.



The lifecycle of neural network verification

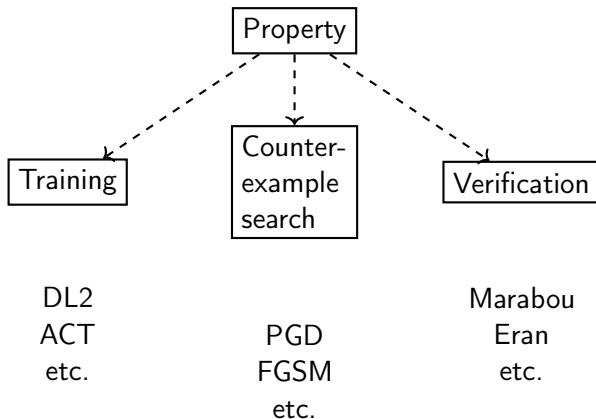


DL2
ACT
etc.

PGD
FGSM
etc.

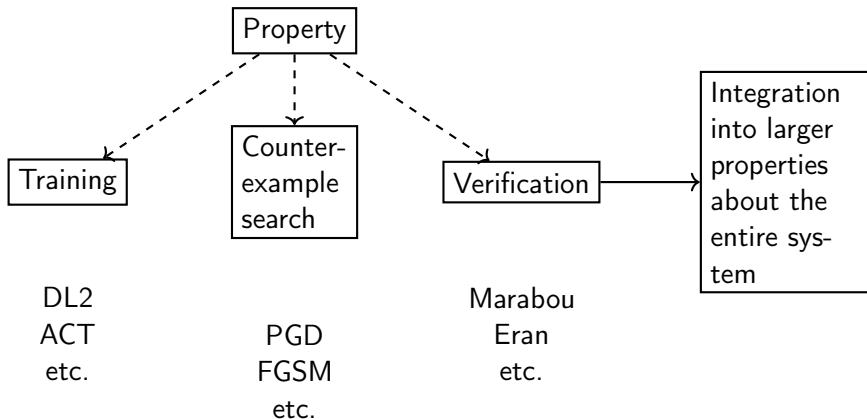


The lifecycle of neural network verification



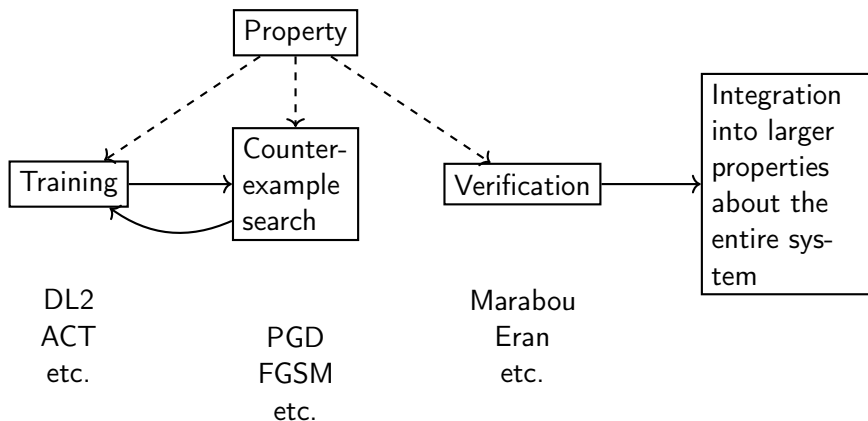


The lifecycle of neural network verification



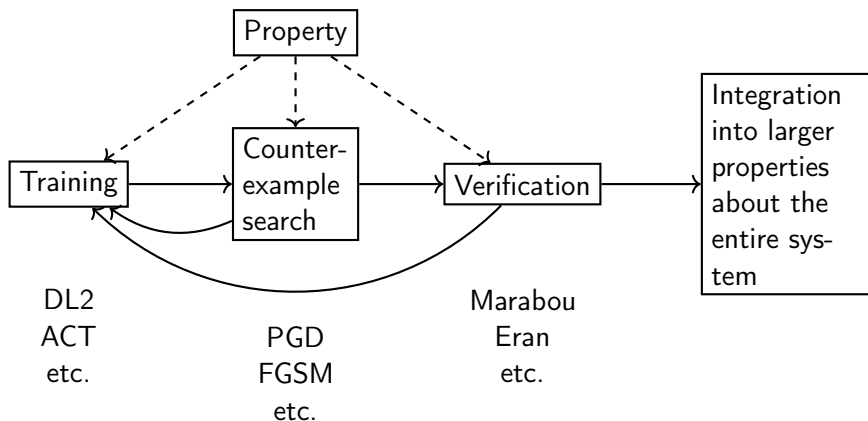


The lifecycle of neural network verification



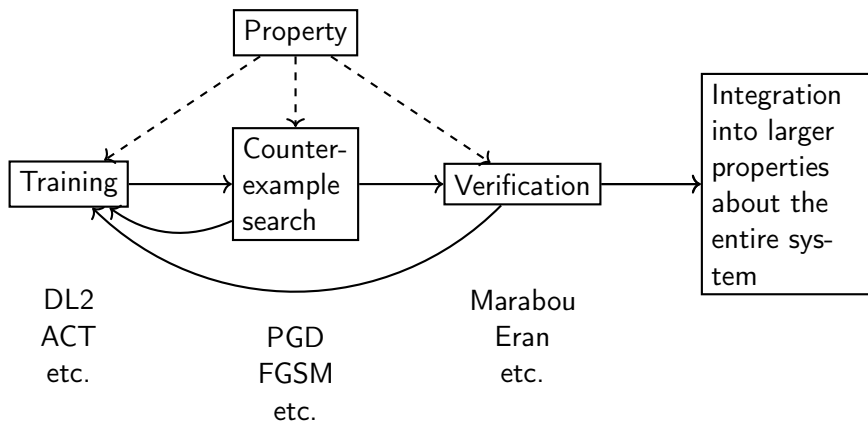


The lifecycle of neural network verification



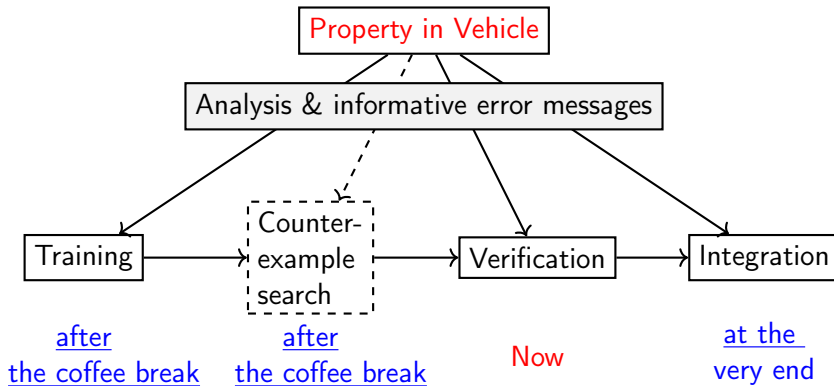


The lifecycle of neural network verification



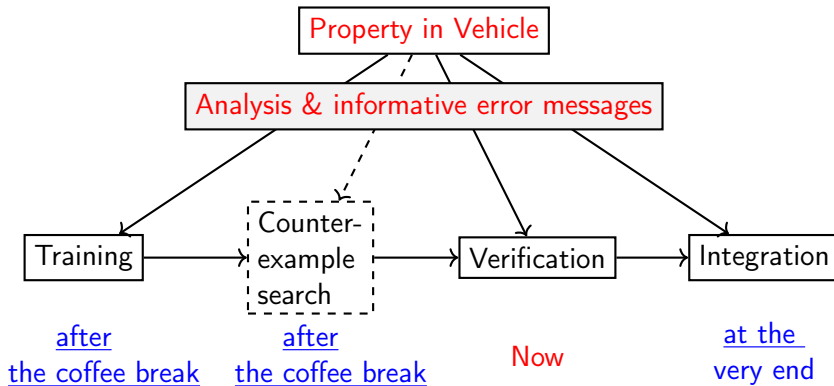


Vehicle: Verification Condition Language



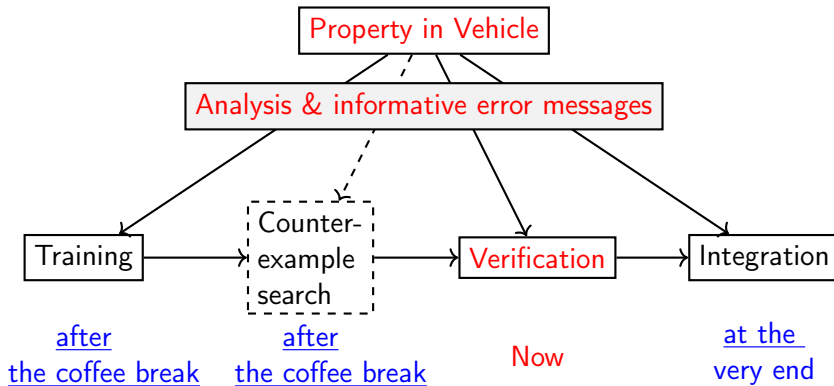


Vehicle: Verification Condition Language



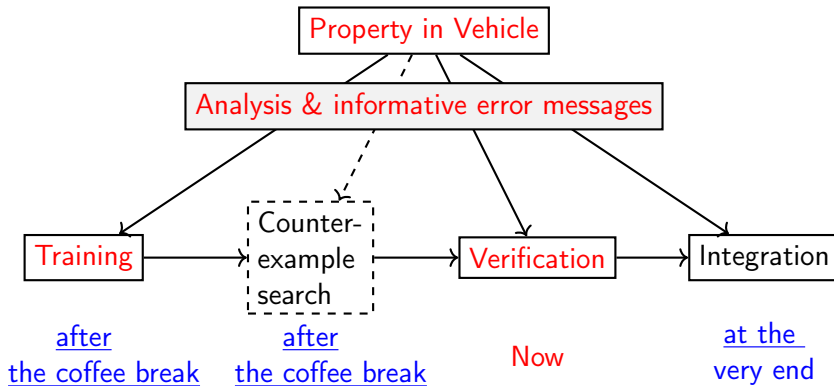


Vehicle: Verification Condition Language



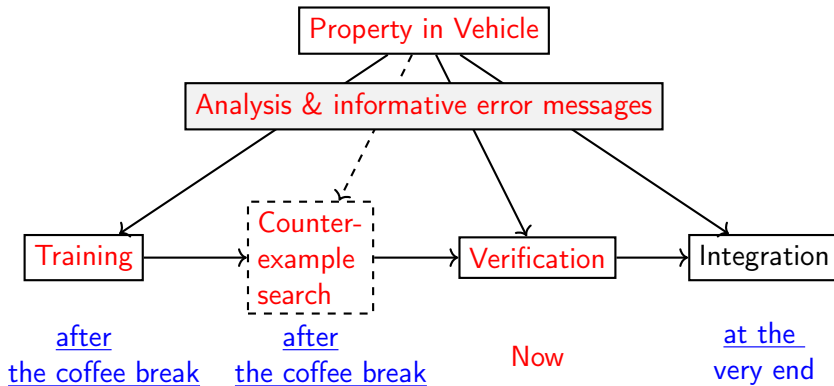


Vehicle: Verification Condition Language



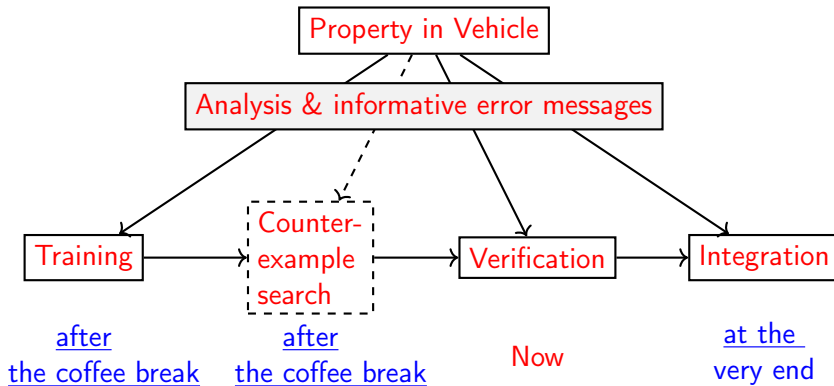


Vehicle: Verification Condition Language





Vehicle: Verification Condition Language





Current Verifier Landscape

A whole range of domain-specific verifiers exist:



Current Verifier Landscape

A whole range of domain-specific verifiers exist:

- ▶ Marabou (SMT solving + Linear Programming)



Current Verifier Landscape

A whole range of domain-specific verifiers exist:

- ▶ Marabou (SMT solving + Linear Programming)
- ▶ ERAN (abstract interpretation + MILP)



Current Verifier Landscape

A whole range of domain-specific verifiers exist:

- ▶ Marabou (SMT solving + Linear Programming)
- ▶ ERAN (abstract interpretation + MILP)
- ▶ Verisig (interval arithmetic)
- ▶ AlphaBetaCROWN (linear bound propagation)

International Standards and Competitions

<https://www.vnnlib.org/>

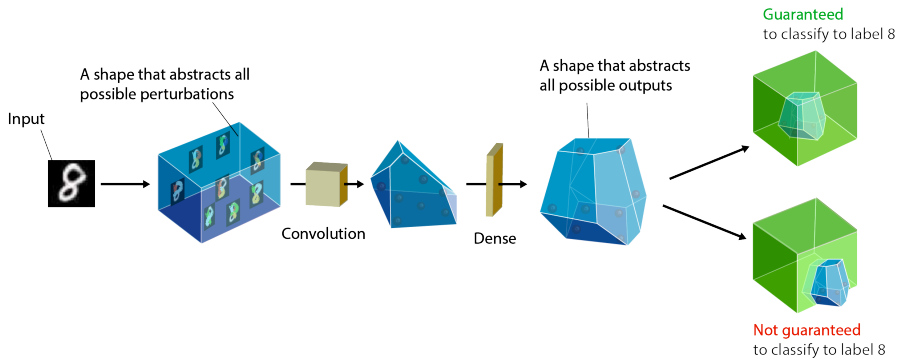
Marabou



H. Wu, O. Isac, A. Zeljic, T. Tagomori, M. L. Daggett, W. Kokke, I. Refaeli, G. Amir, K. Julian, S. Bassan, P. Huang, O. Lahav, M. Wu, M. Zhang, E. Komendantskaya, G. Katz, C. W. Barrett: Marabou 2.0: A Versatile Formal Analyzer of Neural Networks. CAV 2024.



Basic Intuition



ERAN and PRIMA



M.N. Müller, G. Makarchuk, G. Singh, M. Püschel, M. Vechev. PRIMA: Precise and General Neural Network Certification via Multi-Neuron Convex Relaxations. POPL 2022.



Outline

ML in Robotics: What and Why we verify? Importance of Specification

Complexity of AI Verification: ML, Verification, Cyber-Physical Systems Perspectives

The "Hello World" example in ML Verification: the ACAS Xu benchmark

Specification-driven training



Next...

We will:

1. understand how to program the ACAS Xu benchmark in **Vehicle**



Next...

We will:

1. understand how to program the ACAS Xu benchmark in **Vehicle**
2. how to verify the property



ACAS Xu

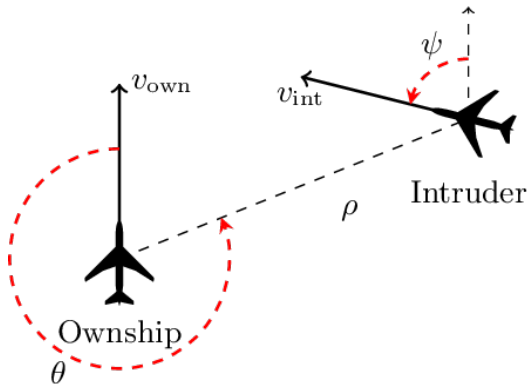
A collision avoidance system for unmanned autonomous aircraft.

Inputs:

- ▶ Distance to intruder, ρ
- ▶ Angle to intruder, θ
- ▶ Intruder heading, φ
- ▶ Speed, v_{own}
- ▶ Intruder speed, v_{int}

Outputs:

- ▶ Clear of conflict
- ▶ Strong left
- ▶ Weak left
- ▶ Weak right
- ▶ Strong right





ACAS Xu

Definition (ACAS Xu: Property 3)

If the intruder is directly ahead and is moving towards the ownship, the score for COC will not be minimal.

$$\begin{aligned} 1500 \leq \rho \leq 1800 \wedge -0.06 \leq \theta \leq 0.06 \wedge \psi \geq 3.10 \wedge v_{own} \geq 980 \wedge v_{int} \geq 960 \\ \Rightarrow \\ \exists a \in \{SL, L, R, SR\}. f(\theta, \rho, \varphi, v_{own}, v_{int})_a < f(\theta, \rho, \varphi, v_{own}, v_{int})_{COC} \end{aligned}$$



Mind the GAP!!!

The Embedding Gap

refers to the case when:

- ▶ our property is specified by the domain experts in units that have certain meaning: meters, radians, etc



Mind the GAP!!!

The Embedding Gap

refers to the case when:

- ▶ our property is specified by the domain experts in units that have certain meaning: meters, radians, etc
- ▶ but:
 1. neural net is normalised for efficiency and performance reasons, and actually operates with different numbers
 2. it is unaware of that designated meaning and returns values that make no sense



Mind the GAP!!!

The Embedding Gap

refers to the case when:

- ▶ our property is specified by the domain experts in units that have certain meaning: meters, radians, etc
- ▶ but:
 1. neural net is normalised for efficiency and performance reasons, and actually operates with different numbers
 2. it is unaware of that designated meaning and returns values that make no sense
- ▶ We want to verify the property stated in original units, but the object we verify – the neural net – does not know about those units



Mind the GAP!!!

The Embedding Gap Exercise

- ▶ Think about examples of neural network applications: in which the embedding gap is likely to be severe, and in which – not?





Mind the GAP!!!

The Embedding Gap Exercise

- ▶ Think about examples of neural network applications: in which the embedding gap is likely to be severe, and in which – not?

... think conversational agents ...





Types

Let us build the ACAS Xu specification.

We start with types of input and output vectors, as well as types of ACAS Xu networks

```
type Input = Tensor Real [5]  
type Output = Tensor Real [5]
```

```
@network  
acasXu : Input -> Output
```

The Vector type represents a mathematical vector, or in programming terms can be thought of as a fixed-length array.



Values

Types for values are automatically inferred by **Vehicle**. For example, we can declare the number π and its type will be inferred as rational:

```
pi = 3.141592
```



Working with vectors

Problem: The trained ACAS Xu network assumes that the inputs and outputs are normalised to values (roughly) between -2 and 2.



Working with vectors

Problem: The trained ACAS Xu network assumes that the inputs and outputs are normalised to values (roughly) between -2 and 2.

However, we want to write our specifications over the original units! (e.g. m/s)



Working with vectors

Problem: The trained ACAS Xu network assumes that the inputs and outputs are normalised to values (roughly) between -2 and 2.

However, we want to write our specifications over the original units! (e.g. m/s)

This is an instance of the common problem space / input space mismatch



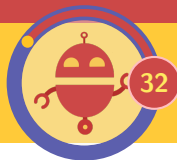
Working with vectors

Problem: The trained ACAS Xu network assumes that the inputs and outputs are normalised to values (roughly) between -2 and 2.

However, we want to write our specifications over the original units! (e.g. m/s)

This is an instance of the common problem space / input space mismatch

Being able to write specifications about the problem space is a feature that distinguishes **Vehicle** from other neural network verifiers platforms.



Vector normalisation

For clarity, we define a new type synonym for unnormalised input vectors which are in the problem space.

```
type UnnormalisedInput = Tensor Real [5]
```

Next we define the range of the inputs that the network is designed to work over.

```
minimumInputValues : UnnormalisedInput  
minimumInputValues = [0, -pi, -pi, 0, 0]
```

```
maximumInputValues : UnnormalisedInput  
maximumInputValues = [60261.0, pi, pi, 1200.0, 1200.0]
```

```
meanScalingValues : UnnormalisedInput  
meanScalingValues = [19791.091, 0.0, 0.0, 650.0, 600.0]
```




Vector manipulation

An alternative method to vector definition is to use the 'foreach' constructor, which is used to provide a value for each 'index i'.

```
minimumInputValues : UnnormalisedInput  
minimumInputValues = foreach i . 0
```

Let us see how 'foreach' works with vector indexing.

We can now define the normalisation function that takes an input vector and returns the unnormalised version.

```
normalise : UnnormalisedInput -> Input  
normalise x = foreach i .  
  (x ! i - meanScalingValues ! i) / (maximumInputValues ! i - minimumInputValues ! i)
```



Vector manipulation

An alternative method to vector definition is to use the 'foreach' constructor, which is used to provide a value for each 'index i'.

```
minimumInputValues : UnnormalisedInput  
minimumInputValues = foreach i . 0
```

Let us see how 'foreach' works with vector indexing.

We can now define the normalisation function that takes an input vector and returns the unnormalised version.

```
normalise : UnnormalisedInput -> Input  
normalise x = foreach i .  
  (x ! i - meanScalingValues ! i) / (maximumInputValues ! i - minimumInputValues ! i)
```

... our first acquaintance with functions!



Functions and types

`<name> : <type>`

`<name> [<args>] = <expr>`

Functions make up the backbone of the **Vehicle** language.



Functions and types

```
<name> : <type>  
<name> [<args>] = <expr>
```

Functions make up the backbone of the **Vehicle** language.

```
validInput : UnnormalisedInput -> Bool  
validInput x = forall i .  
  minimumInputValues ! i <= x ! i <= maximumInputValues ! i
```



Functions and types

```
<name> : <type>  
<name> [<args>] = <expr>
```

Functions make up the backbone of the **Vehicle** language.

```
validInput : UnnormalisedInput -> Bool  
validInput x = forall i .  
  minimumInputValues ! i <= x ! i <= maximumInputValues ! i
```

Our first acquaintance with predicates and quantifiers!

One of the main advantages of **Vehicle** is that it can be used to state and prove specifications that describe the network's behaviour over an infinite set of values.



Functions and types

Function Composition: Exercise

What are the types of functions 'acasXu' and 'normalise':

```
normAcasXu : UnnormalisedInput -> Output
```

```
normAcasXu x = acasXu (normalise x)
```



Let's verify ACAS Xu!

```
distanceToIntruder = 0    -- measured in metres
angleToIntruder    = 1    -- measured in radians
intruderHeading    = 2    -- measured in radians
speed              = 3    -- measured in metres/second
intruderSpeed      = 4    -- measured in meters/second
```

```
clearOfConflict = 0
weakLeft       = 1
weakRight      = 2
strongLeft     = 3
strongRight    = 4
```

The fact that all vector types come annotated with their size means that it is impossible to mess up indexing into vectors, e.g. if you changed 'distanceToIntruder = 0' to 'distanceToIntruder = 5' the specification would fail to type-check.



Property 3

If the intruder is directly ahead and is moving towards the ownship, the score for COC will not be minimal.



Property 3

If the intruder is directly ahead and is moving towards the ownship, the score for COC will not be minimal.

```
directlyAhead : UnnormalisedInput -> Bool
directlyAhead x =
  1500 <= x ! distanceToIntruder <= 1800 and
  -0.06 <= x ! angleToIntruder <= 0.06
```



Property 3

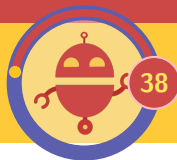
If the intruder is directly ahead and is moving towards the ownship, the score for COC will not be minimal.

```
directlyAhead : UnnormalisedInput -> Bool
directlyAhead x =
  1500 <= x ! distanceToIntruder <= 1800 and
  -0.06 <= x ! angleToIntruder <= 0.06
```

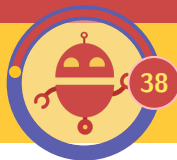
Exercise!

1. Can you identify whether the specification is written in terms of input space or problem space?
How do you know?

Property 3



If the intruder is directly ahead and is moving towards the ownship, the score for COC will not be minimal.



Property 3

If the intruder is directly ahead and is moving towards the ownship, the score for COC will not be minimal.

```
movingTowards : UnnormalisedInput -> Bool
movingTowards x =
  x ! intruderHeading >= 3.10  and
  x ! speed            >= 980   and
  x ! intruderSpeed    >= 960
```



There is little left to do!

If the intruder is directly ahead and is moving towards the ownship, the score for COC will not be minimal.



There is little left to do!

If the intruder is directly ahead and is moving towards the ownship, the score for COC will not be minimal.

```
minimalScore : Index 5 -> UnnormalisedInput -> Bool
minimalScore i x =
  forall j . i != j => normAcasXu x ! i < normAcasXu x ! j
```



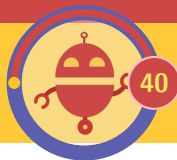
There is little left to do!

If the intruder is directly ahead and is moving towards the ownship, the score for COC will not be minimal.

```
minimalScore : Index 5 -> UnnormalisedInput -> Bool
minimalScore i x =
  forall j . i != j => normAcasXu x ! i < normAcasXu x ! j
```

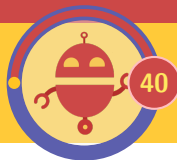
Exercise!

1. What kind of domain 'forall' ranges over? Is it finite or infinite?



There is little left to do!

If the intruder is directly ahead and is moving towards the ownship, the score for COC will not be minimal.



There is little left to do!

If the intruder is directly ahead and is moving towards the ownship, the score for COC will not be minimal.

```
@property
property3 : Bool
property3 = forall x . validInput x and
                      directlyAhead x and
                      movingTowards x =>
                      not (minimalScore clearOfConflict x)
```

Exercise!

1. Can you guess the purpose of the '@property' syntax?
2. What kind of domain 'forall' ranges over? Is it finite or infinite?



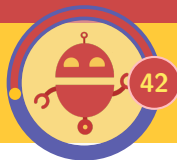
A technical note

In the exercise session, you will have:

```
advises : Index 5 -> UnnormalisedInput -> Bool
advises i x = forall j . i != j => normAcasXu x ! i < normAcasXu x ! j
```

```
@property
property3 : Bool
property3 = forall x . validInput x and
                        directlyAhead x and
                        movingTowards x =>
                        not (advises clearOfConflict x)
```

The `advises` function is equivalent to the `minimalScore` function, but more generally applicable in other cases.



Setup

- ▶ Download the tutorial repository:

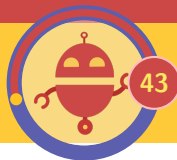
```
git clone https://github.com/vehicle-lang/tutorial
```

- ▶ Install Vehicle:

```
pip install vehicle-lang
```

- ▶ Install Marabou verifier:

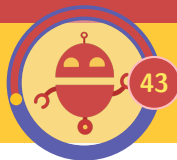
```
pip install maraboupy
```



How to run Vehicle

Checklist

1. Vehicle and Marabou are installed.
2. navigate to `examples/acasXu` in the tutorial repo.



How to run Vehicle

Checklist

1. Vehicle and Marabou are installed.
2. navigate to examples/acasXu in the tutorial repo.

```
vehicle verify \  
  --specification acasXu.vcl \  
  --verifier Marabou \  
  --network acasXu:acasXu_1_7.onnx \  
  --property property3
```

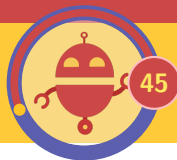
Verifying properties:

```
property3 [=====] 1/1 queries  
result: counterexample found  
x: [1799.9886669999978, 1.9509286320000003e-2,  
    3.09999732192, 980.0, 1017.6036]
```



Sources

-  M. Daggit, A. Sirman, A. Bruni, S. Teuber, E. Komendantskaya, J. Smart, G. Passmore. Compositional Neural-Cyber-Physical System Verification in the Interactive Theorem Prover of Your Choice. ICFP 2026
-  M. Daggett, W. Kokke, R. Atkey, E. Komendantskaya, N. Slusarz and L. Arnaboldi. Vehicle: Bridging the Embedding Gap in the Verification of Neuro-Symbolic Programs. FSCD'25
-  Vehicle Team: The Vehicle language: <https://github.com/vehicle-lang> 2026.
-  M. Daggett and W. Kokke: Vehicle User Manual. <https://vehicle-lang.readthedocs.io> 2026.
-  M. Daggett, W. Kokke, E. Komendantskaya, R. Atkey, L. Arnaboldi, N. Slusarz, M. Casadio, B. Coke, and J. Lee. The Vehicle Tutorial: Neural Network Verification with Vehicle. The 6th Workshop on Formal Methods for ML-Enabled Autonomous Systems (FoMLAS'23).



Verification tutorials: Vehicle



1 Introduction (5)

2 Getting Started with the
Vehicle Specification Language
(17)

3 Proving Neural Network
Robustness (15)

4 Property-Driven Training (10)

5 Exporting to Agda (2)

6 Conclusions and Future
Directions

References

1 Introduction

Welcome to the *Vehicle* tutorial page! In this tutorial, you will learn about neural network verification with *Vehicle*. *Vehicle*'s complete documentation and installation instructions are available on the [vehicle documentation](#) page.

The present tutorial contains contributions by:

- Matthew Daggitt
- Wen Kokke
- Ekaterina Komendantskaya
- Bob Atkey
- Luca Arnaboldi
- Natalia Slusarz
- Marco Casadio
- Ben Coke
- Jeonghyeon Lee

This tutorial has been run live and adapted for different audiences at:

- [FOMLAS'23 the 6th Workshop on Formal Methods for ML-Enabled Autonomous Systems](#) co-allocated with CAV'23 in Paris, France; 18 July 2023;
- [VETSS Summer School](#) at the University of Surrey, UK; 23 August 2023;



Vehicle Tutorial

- ▶ Vehicle is designed to make specifications of neural networks simpler (for users outside of formal methods community)
- ▶ Vehicle Tutorial has been presented at CAV'23, VETSS Summer School'23, ICFP'23
- ▶ and in 2025: at the UK Robotics Summer School and SAT/SMT/AR Summer School

 The Vehicle team: Vehicle Tutorial <https://github.com/vehicle-lang/vehicle-tutorial>.



Harder Exercise: ACAS Xu Property 1

ACAS Xu Property 1 gives an idea how the *embedding gap* can arise not only when we reason about inputs, but also the outputs of networks!



Harder Exercise: ACAS Xu Property 1

ACAS Xu Property 1 gives an idea how the *embedding gap* can arise not only when we reason about inputs, but also the outputs of networks!

Definition: ACAS Xu: Property 1

If the intruder is distant and is significantly slower than the ownship, the score of a COC advisory will always be below a certain fixed threshold:

$$(\rho \geq 55947.691) \wedge (v_{own} \geq 1145) \wedge (v_{int} \leq 60) \\ \Rightarrow \text{the score for COC is at most 1500}$$

where the neural network outputs are scaled as follows: given an element x of the output vector, we scale it as: $\frac{x-7.518884}{373.94992}$.



Harder Exercise: ACAS Xu Property 1

ACAS Xu Property 1 gives an idea how the *embedding gap* can arise not only when we reason about inputs, but also the outputs of networks!

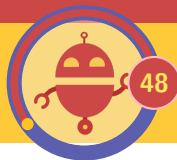
Definition: ACAS Xu: Property 1

If the intruder is distant and is significantly slower than the ownship, the score of a COC advisory will always be below a certain fixed threshold:

$$(\rho \geq 55947.691) \wedge (v_{own} \geq 1145) \wedge (v_{int} \leq 60) \\ \Rightarrow \text{the score for COC is at most 1500}$$

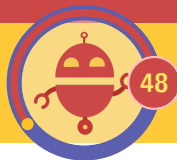
where the neural network outputs are scaled as follows: given an element x of the output vector, we scale it as: $\frac{x-7.518884}{373.94992}$.

- ▶ Can you formalise Property 1 in Vehicle?
- ▶ Can you spot the embedding gap, this time concerning the network's output?



Conclusions

- ▶ Today, we started with the idea of a "property" we want the system (neural network) to satisfy
- ▶ We discussed different efforts that various communities made in order to:
 - ▶ state/specify/formally define the properties
 - ▶ ... implement programming languages where property specification is possible
 - ▶ implement programs (=solvers) that can say with mathematical certainty whether those properties hold or not



Conclusions

- ▶ Today, we started with the idea of a "property" we want the system (neural network) to satisfy
- ▶ We discussed different efforts that various communities made in order to:
 - ▶ state/specify/formally define the properties
 - ▶ ... implement programming languages where property specification is possible
 - ▶ implement programs (=solvers) that can say with mathematical certainty whether those properties hold or not
- ▶ We looked closer into one of these languages, Vehicle:
 - ▶ got familiar with its syntax
 - ▶ wrote and verified a specification



Big Picture

Big Picture

- ▶ Initially, there was no notion of property specification in ML, Data was "everything"



Big Picture

Big Picture

- ▶ Initially, there was no notion of property specification in ML, Data was "everything"
- ▶ Ultimately, ML researchers realised that properties of interest (separate from data) do exist (famous example: robustness to attack)



Big Picture

Big Picture

- ▶ Initially, there was no notion of property specification in ML, Data was "everything"
- ▶ Ultimately, ML researchers realised that properties of interest (separate from data) do exist (famous example: robustness to attack)
- ▶ At which point, they only had languages suitable for optimisation, not verification. Bespoke ML verifiers had to be developed.



Big Picture

Big Picture

- ▶ Initially, there was no notion of property specification in ML, Data was "everything"
- ▶ Ultimately, ML researchers realised that properties of interest (separate from data) do exist (famous example: robustness to attack)
- ▶ At which point, they only had languages suitable for optimisation, not verification. Bespoke ML verifiers had to be developed.
- ▶ Current state of the art: looking for ways to get the best of both worlds:
 - ▶ still have clever ML algorithms,
 - ▶ supplement them with verification when needed;
 - ▶ design languages that will enable to program in (or with) "both worlds".



Outline

ML in Robotics: What and Why we verify? Importance of Specification

Complexity of AI Verification: ML, Verification, Cyber-Physical Systems Perspectives

The "Hello World" example in ML Verification: the ACAS Xu benchmark

Specification-driven training



In this chapter...

We will discuss:

- ▶ ... why and how training is a part of verification of neural networks



In this chapter...

We will discuss:

- ▶ ... why and how training is a part of verification of neural networks
- ▶ ... what choices exist for implementing this, generally



In this chapter...

We will discuss:

- ▶ ... why and how training is a part of verification of neural networks
- ▶ ... what choices exist for implementing this, generally
- ▶ ... what choices **Vehicle** makes in this respect



In this chapter...

We will discuss:

- ▶ ... why and how training is a part of verification of neural networks
- ▶ ... what choices exist for implementing this, generally
- ▶ ... what choices **Vehicle** makes in this respect
- ▶ ... theoretical and practical issues with the chosen methods, and **Vehicle**'s take on them



Why Training is a part of Verification?

```
vehicle verify \  
  --specification acasXu.vcl \  
  --verifier Marabou \  
  --network acasXu:acasXu_1_7.onnx \  
  --property property3
```

Verifying properties:

```
property3 [=====] 1/1 queries  
result: counterexample found  
x: [1799.9886669999978, 1.9509286320000003e-2,  
    3.09999732192, 980.0, 1017.6036]
```




A few words on the context

1943 Perceptron by McCulloch and Pitts

90-2000 Rise of machine learning applications

2013  C. Szegedy, W. Zaremba, I. Sutskever, J. Bruna, D. Erhan, I. Goodfellow, and R. Fergus. Intriguing properties of neural networks. 2013. (10000+ citations on GS)

2013-.. Tens of thousands of papers on adversarial training
(in the attack-defence style)

 A. C. Serban, E. Poll, J. Visser. Adversarial Examples - A Complete Characterisation of the Phenomenon. 2019.



A few words on the context

1943 Perceptron by McCulloch and Pitts

90-2000 Rise of machine learning applications

2013  C. Szegedy, W. Zaremba, I. Sutskever, J. Bruna, D. Erhan, I. Goodfellow, and R. Fergus. Intriguing properties of neural networks. 2013. (10000+ citations on GS)

2013-.. Tens of thousands of papers on adversarial training
(in the attack-defence style)

 A. C. Serban, E. Poll, J. Visser. Adversarial Examples - A Complete Characterisation of the Phenomenon. 2019.

2017 First Neural network verification attempts

 G. Katz, C.W. Barrett, D.L. Dill, K. Julian, M.J. Kochenderfer: Reluplex: An Efficient SMT Solver for Verifying Deep Neural Networks. CAV (1) 2017: 97-117.

 X. Huang, M. Kwiatkowska, S. Wang, M. Wu. Safety Verification of Deep Neural Networks. CAV (1) 2017: 3-29.

2017-.. Hundreds of papers on neural network verification



Training for Robustness

Training generally:

1. depends on data
2. depends on loss functions
3. some other parameters like shape of the functions



1. Data Augmentation

Suppose we are given a data set $\mathcal{D} = \{(x_1, y_1), \dots, (x_n, y_n)\}$.

Prior to training, generate new training data samples close to existing data and label them with the same output as the original data.

 C. Shorten, T.M. Khoshgoftaar: A survey on image data augmentation for deep learning. J. Big Data 6, 60 (2019)

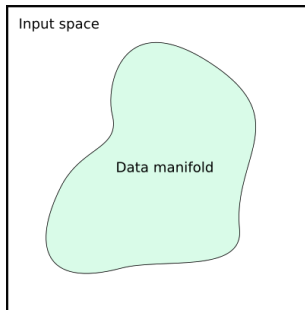


1. Data Augmentation

Suppose we are given a data set $\mathcal{D} = \{(x_1, y_1), \dots, (x_n, y_n)\}$.

Prior to training, generate new training data samples close to existing data and label them with the same output as the original data.

 C. Shorten, T.M. Khoshgoftaar: A survey on image data augmentation for deep learning. J. Big Data 6, 60 (2019)

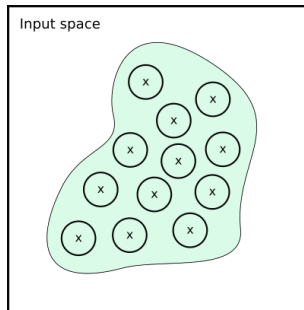




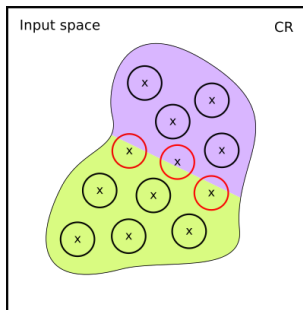
1. Data Augmentation

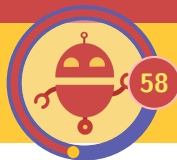
Suppose we are given a data set $\mathcal{D} = \{(x_1, y_1), \dots, (x_n, y_n)\}$. Prior to training, generate new training data samples close to existing data and label them with the same output as the original data.

 C. Shorten, T.M. Khoshgoftaar: A survey on image data augmentation for deep learning. J. Big Data 6, 60 (2019)

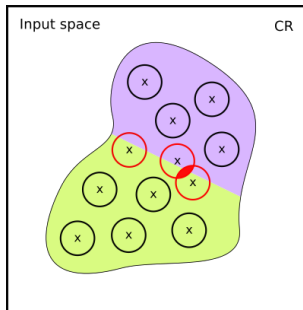


However,



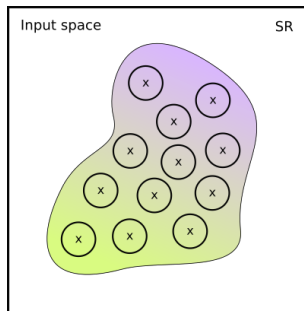


However,





Adversarial Training



I.J. Goodfellow, J. Shlens, C. Szegedy: Explaining and harnessing adversarial examples. 3rd International Conference on Learning Representations, ICLR 2015, San Diego, CA, USA, May 7-9, 2015, Conference Track Proceedings (2015)



2. Solutions Involving Loss Functions

Given a data set $\mathcal{D}$,

a function $f_\theta : \mathbb{R}^n \rightarrow \mathbb{R}^m$ (with optimisation parameters θ),

a loss function $\mathcal{L} : \mathbb{R}^n \times \mathbb{R}^m \rightarrow \mathbb{R}$ computes a penalty proportional to the difference between the output of f_θ on a training input $\hat{x}$ and a desired output y .



2. Solutions Involving Loss Functions

Given a data set $\mathcal{D}$,

a function $f_\theta : \mathbb{R}^n \rightarrow \mathbb{R}^m$ (with optimisation parameters θ),

a loss function $\mathcal{L} : \mathbb{R}^n \times \mathbb{R}^m \rightarrow \mathbb{R}$ computes a penalty proportional to the difference between the output of f_θ on a training input $\hat{x}$ and a desired output y .

Example (Cross Entropy Loss Function)

Given a function $f_\theta : \mathbb{R}^n \rightarrow [0, 1]^m$, the cross-entropy loss is defined as

$$\mathcal{L}_{ce}(\hat{x}, y) = - \sum_{i=1}^m y_i \log(f_\theta(\hat{x})_i) \quad (1)$$

where y_i is the true probability for class i and $f_\theta(\hat{x})_i$ is the probability for class i as predicted by f_θ when applied to $\hat{x}$.



2. Adversarial Training for Robustness

- ▶ gradient descent minimises loss $\mathcal{L}(\hat{x}, y)$ between the predicted value $f_{\theta}(\hat{x})$ and the true value y , for each entry $(\hat{x}, y)$ in $\mathcal{D}$. It thus solves the optimisation problem:

$$\min_{\theta} \mathcal{L}(\hat{x}, y)$$



2. Adversarial Training for Robustness

- ▶ gradient descent minimises loss $\mathcal{L}(\hat{x}, y)$ between the predicted value $f_{\theta}(\hat{x})$ and the true value y , for each entry $(\hat{x}, y)$ in $\mathcal{D}$. It thus solves the optimisation problem:

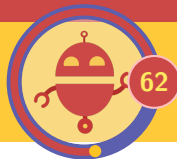
$$\min_{\theta} \mathcal{L}(\hat{x}, y)$$

- ▶ For adversarial training, we instead minimise the loss with respect to the worst-case perturbation of each sample in $\mathcal{D}$.
 - ▶ Replace the standard training objective with:

$$\min_{\theta} \left[\max_{x: |x - \hat{x}| \leq \epsilon} \mathcal{L}(x, y) \right]$$

- ▶ the inner maximisation is done by "projected gradient descent" (PGD), that "projects" the gradient of $\mathcal{L}$ on $\hat{x}$ in order to perturb it and get the worst x .





Lipshitz Continuity

Optimise for:

$$\forall \mathbf{x} : |\mathbf{x} - \hat{\mathbf{x}}| \leq \epsilon \Rightarrow |f(\mathbf{x}) - f(\hat{\mathbf{x}})| \leq L|\mathbf{x} - \hat{\mathbf{x}}|$$

-  P. Pauli, A. Koch, J. Berberich, P. Kohler, F. Allgower: Training robust neural networks using Lipschitz bounds. IEEE Control Systems Letters (2021)
-  H. Gouk, E. Frank, B. Pfahringer, M.J. Cree: Regularisation of neural networks by enforcing Lipschitz continuity. Machine Learning 110(2), 393–416 (2021)

and much more...



3. Other ways to do property-driven training

More sophisticated approaches to property driven training include:

- ▶ Tailoring the neural network architectures
- ▶ Tailoring the activation functions
- ▶ Including probabilistic or deterministic solvers into neural network layers
- ▶ ...

 Eleonora Giunchiglia, Mihaela Catalina Stoian, and Thomas Lukasiewicz. Deep learning with logical constraints. IJCAI 2022, pages 5478–5485. ijcai.org, 2022

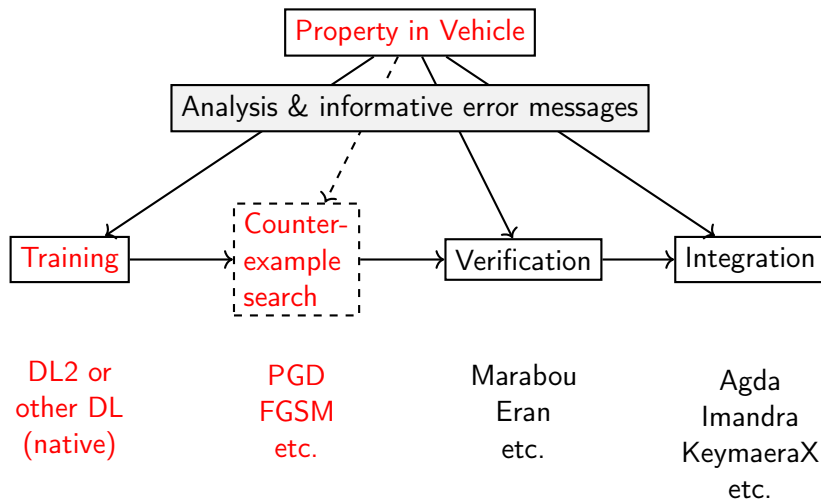


The solution we are looking for





In Vehicle terms,





Vehicle's formula

Property-driven training =
Differentiable Logic + (PGD) optimisation



Example - differentiable logic

We define a very simple differentiable logic on a toy language

$$p := p \mid a \leq a \mid p \wedge p$$

based on Capucci logic [Capucci et al 2026].

$$\mathsf{T}(a \leq b) := a - b$$

$$\mathsf{T}(a \wedge b) := -\frac{1}{p} \log(e^{p\mathsf{T}(a)} + e^{p\mathsf{T}(b)})$$



Different existing DLs

► DL2



Marc Fischer, Mislav Balunovic, Dana Drachsler-Cohen, Timon Gehr, Ce Zhang, and Martin Vechev. DL2: Training and querying neural networks with logic. In ICML'19, pp. 1931–1941.

► Signal Temporal Logic DL



Peter Varnai and Dimos V. Dimarogonas. On robustness metrics for learning stl tasks. In 2020 American Control Conference (ACC), pp. 5394–5399, 2020.

► fuzzy DLs such as: Godel, Łukasiewicz, Yager, product and others



Emile van Krieken, Erman Acar, and Frank van Harmelen. Analyzing differentiable fuzzy logic operators. Artificial Intelligence, 302:103602, 2022.

► Quantitative Linear Logic

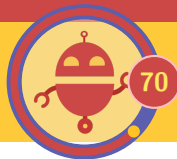


T Flinkow, E Komendantskaya, M Capucci, R Monahan. Quantitative Linear Logic for Neuro-Symbolic Learning and Verification. arXiv preprint arXiv:2605.13845. 2026



Vehicle's formula

Property-driven training =
Differentiable Logic + (PGD) optimisation

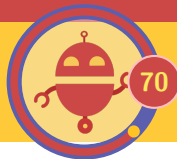


Optimisation

Recall: With “projected gradient descent” (PGD),
we minimise the loss wrt the worst-case perturbation of each sample in $\mathcal{D}$.

- Replace the standard training objective with:

$$\min_{\theta} \left[\max_{x: |x - \hat{x}| \leq \epsilon} \mathcal{L}(x, y) \right]$$



Optimisation

Recall: With “projected gradient descent” (PGD), we minimise the loss wrt the worst-case perturbation of each sample in $\mathcal{D}$.

- Replace the standard training objective with:

$$\min_{\theta} \left[\max_{x: |x - \hat{x}| \leq \epsilon} \mathcal{L}(x, y) \right]$$

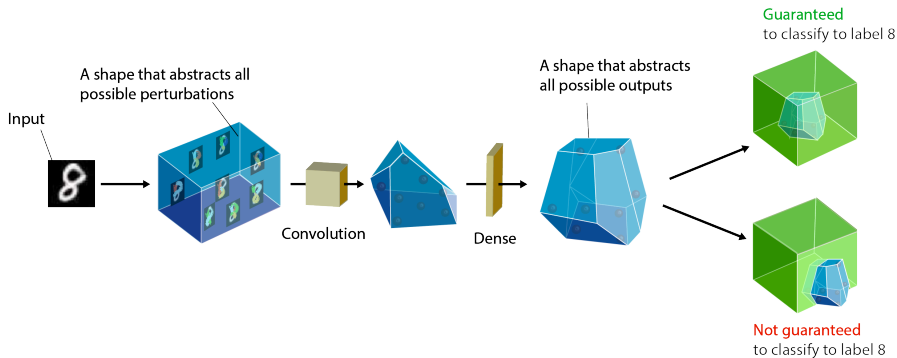
- in Vehicle, given a property $\forall x. \mathcal{P}(x) \Rightarrow \mathcal{S}(x)$, we replace the above training objective with

$$\min_{\theta} \left[\max_{x \in \mathbb{H}_{\mathcal{P}(x)}} \mathcal{L}_{\mathcal{S}(x)}(x, y) \right]$$

where $\mathbb{H}_{\mathcal{P}(x)}$ is a hyper-shape (usually a hyper-rectangle) that corresponds to the pre-condition $\mathcal{P}(x)$ and $\mathcal{L}_{\mathcal{S}(x)}$ is obtained by DL-translation of the post-condition $\mathcal{S}(x)$.



Recall



ERAN and PRIMA

 M.N. Müller, G. Makarchuk, G. Singh, M. Püschel, M. Vechev. PRIMA: Precise and General Neural Network Certification via Multi-Neuron Convex Relaxations. POPL 2022.



Examples: AcasXU

```
@property
property3 : Bool
property3 = forall x . validInput x and
                    directlyAhead x and
                    movingTowards x =>
                    not (minimalScore clearOfConflict x)
```

- $\mathbb{H}_{\mathcal{P}(x)}$ is given by (the normalised) vector bounds given by:

$$1500 \leq \rho \leq 1800$$

$$-0.06 \leq \theta \leq 0.06$$

$$\psi \geq 3.10$$

$$v_{own} \geq 980$$

$$v_{int} \geq 960$$

- Recall that for Acas Xu networks, $x = [\rho, \theta, \psi, v_{own}, v_{int}]^{norm}$



Examples: AcasXU

```
@property
property3 : Bool
property3 = forall x . validInput x and
                    directlyAhead x and
                    movingTowards x =>
                    not (minimalScore clearOfConflict x)
```

- $\mathbb{H}_{\mathcal{P}(x)}$ is given by (the normalised) vector bounds given by:

$$1500 \leq \rho \leq 1800$$

$$-0.06 \leq \theta \leq 0.06$$

$$\psi \geq 3.10$$

$$v_{own} \geq 980$$

$$v_{int} \geq 960$$

- Recall that for Acas Xu networks, $x = [\rho, \theta, \psi, v_{own}, v_{int}]^{norm}$
- and $\mathcal{L}_{\mathcal{S}(x)} = \text{T}(\text{not } (\text{minimalScore clearOfConflict } x))$.



Examples: Degenerate case

- ▶ When we have

$$\forall x. \mathcal{S}(x)$$

instead of $\forall x. \mathcal{P}(x) \Rightarrow \mathcal{S}(x)$

Recall we solve the optimisation problem

$$\min_{\theta} \left[\max_{x \in \mathbb{H}_{\mathcal{P}(x)}} \mathcal{L}_{\mathcal{S}(x)}(x, y) \right]$$

- ▶ $\mathbb{H}_{\mathcal{P}(x)} = \mathbb{H}_x$ is the domain of x (usually constrained by normalisation boundaries, e.g. by $[0, 0, \dots, 0]$ and $[1, 1, \dots, 1]$.)



Examples: standard robustness

- ▶ When we have

$$\forall x. |x - \hat{x}| \leq \epsilon \Rightarrow |f(x) - f(\hat{x})| \leq \delta$$

Recall we solve the optimisation problem

$$\min_{\theta} \left[\max_{x \in \mathbb{H}_{\mathcal{P}(x)}} \mathcal{L}_{\mathcal{S}(x)}(x, y) \right]$$

- ▶ $\mathbb{H}_{\mathcal{P}(x)}$ is the ϵ -cube around $\hat{x}$
- ▶ and, in case we use the Product DL, we get
$$\mathcal{L}_{\mathcal{S}(x)} = \mathbb{T}(\|f(x) - f(\hat{x})\| \leq \delta) = |f(x) - f(\hat{x})| - \delta$$



Vehicle's way to implement this

- ▶ You use the same specification ("VCL file") as for verification



Vehicle's way to implement this

- ▶ You use the same specification ("VCL file") as for verification
- ▶ Using provided Python template, you call the specification and the network when running the Python Script:



Vehicle's way to implement this

- ▶ You use the same specification ("VCL file") as for verification
- ▶ Using provided Python template, you call the specification and the network when running the Python Script:
 - ▶ the loss function $\mathcal{L}_{\mathcal{S}(x)}$



Vehicle's way to implement this

- ▶ You use the same specification ("VCL file") as for verification
- ▶ Using provided Python template, you call the specification and the network when running the Python Script:
 - ▶ the loss function $\mathcal{L}_{\mathcal{S}(x)}$
 - ▶ the hyper-shape $\mathbb{H}_{\mathcal{P}(x)}$are automatically generated from the spec!



Vehicle's way to implement this

- ▶ You use the same specification ("VCL file") as for verification
- ▶ Using provided Python template, you call the specification and the network when running the Python Script:
 - ▶ the loss function $\mathcal{L}_{\mathcal{S}(x)}$
 - ▶ the hyper-shape $\mathbb{H}_{\mathcal{P}(x)}$
are automatically generated from the spec!
 - ▶ PGD finds counter-examples within $\mathbb{H}_{\mathcal{P}(x)}$,



Vehicle's way to implement this

- ▶ You use the same specification ("VCL file") as for verification
- ▶ Using provided Python template, you call the specification and the network when running the Python Script:
 - ▶ the loss function $\mathcal{L}_{\mathcal{S}(x)}$
 - ▶ the hyper-shape $\mathbb{H}_{\mathcal{P}(x)}$
are automatically generated from the spec!
 - ▶ PGD finds counter-examples within $\mathbb{H}_{\mathcal{P}(x)}$,
 - ▶ native (SGD) training (in Tensorflow) is used, given the loss function $\mathcal{L}_{\mathcal{S}(x)}$

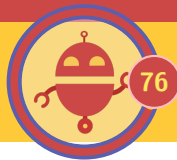


Vehicle's way to implement this

- ▶ You use the same specification ("VCL file") as for verification
- ▶ Using provided Python template, you call the specification and the network when running the Python Script:
 - ▶ the loss function $\mathcal{L}_{\mathcal{S}(\mathbf{x})}$
 - ▶ the hyper-shape $\mathbb{H}_{\mathcal{P}(\mathbf{x})}$
are automatically generated from the spec!
 - ▶ PGD finds counter-examples within $\mathbb{H}_{\mathcal{P}(\mathbf{x})}$,
 - ▶ native (SGD) training (in Tensorflow) is used, given the loss function $\mathcal{L}_{\mathcal{S}(\mathbf{x})}$

...to solve the optimisation problem

$$\min_{\theta} \left[\max_{\mathbf{x} \in \mathbb{H}_{\mathcal{P}(\mathbf{x})}} \mathcal{L}_{\mathcal{S}(\mathbf{x})}(\mathbf{x}, y) \right]$$

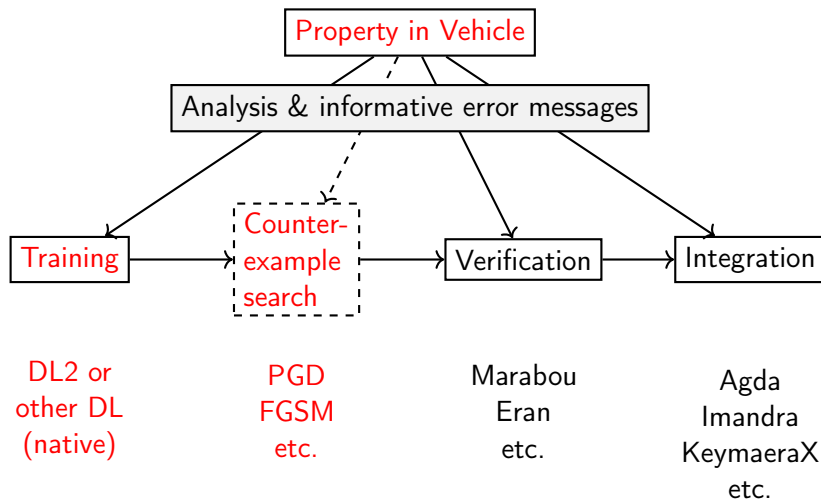


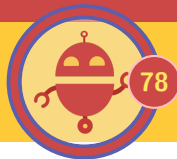
Next:

- ▶ You will get to see ML backend if Vehicle in Action using a case study
- ▶ And in the lab



In Conclusion,





Sources

-  T Flinkow, M Casadio, C Kessler, R Monahan, E Komendantskaya. A generalised framework for property-driven machine learning. arXiv e-prints, arXiv: 2505.00466. 2025
-  M. Daggitt, W. Kokke, R. Atkey, E. Komendantskaya, N. Slusarz and L. Arnaboldi. Vehicle: Bridging the Embedding Gap in the Verification of Neuro-Symbolic Programs. FSCD'25
-  T Flinkow, E Komendantskaya, M Capucci, R Monahan. Quantitative Linear Logic for Neuro-Symbolic Learning and Verification. arXiv preprint arXiv:2605.13845. 2026
-  M.Daggitt and W.Kokke: Vehicle User Manual. <https://vehicle-lang.readthedocs.io> 2026.
-  M. Daggitt, W. Kokke, E. Komendantskaya, R. Atkey, L. Arnaboldi, N. Slusarz, M. Casadio, B. Coke, and J. Lee. The Vehicle Tutorial: Neural Network Verification with Vehicle. The 6th Workshop on Formal Methods for ML-Enabled Autonomous Systems (FoMLAS'23).



PS: DAIR CDT and AI Verification Training at Edinburgh Centre for Robotics

DAIR - Dependable AI for Robotics

We teach 20 hours of lectures (+ 30 hours of labs)

- ▶ Block 1: Introduction to the topic. Safeguarding Machine Learning using Machine Learning methods (Robustness, XAI)
- ▶ Block 2: Neural Network Verification & Property Specification, for stand-alone neural networks
- ▶ Block 3: Verification of dynamical cyber-physical systems (with closed-loop and finite horizon) neural network controllers
- ▶ Block 4: Verification of dynamical cyber-physical systems (with open-loop and infinite horizon) neural network controllers



Applications of Vehicle and NN verification in Robotics

We always look to expand the range of users, collaborators and case studies

► Conversational Agents



M Casadio, T Dinkar, E Komendantskaya, L Arnaboldi, ML Daggitt, O Isac, ... et al. NLP verification: Towards a general methodology for certifying robustness. European Journal of Applied Mathematics. 2025



Applications of Vehicle and NN verification in Robotics

We always look to expand the range of users, collaborators and case studies

► Conversational Agents



M Casadio, T Dinkar, E Komendantskaya, L Arnaboldi, ML Daggitt, O Isac, ... et al. NLP verification: Towards a general methodology for certifying robustness. European Journal of Applied Mathematics. 2025

► Drones



C Kessler, E Komendantskaya, M Casadio, IM Viola, T Flinkow, ... et al. Neural Network Verification for Gliding Drone Control: A Case Study. Symposium on AI Verification: SAIV 2025



Applications of Vehicle and NN verification in Robotics

We always look to expand the range of users, collaborators and case studies

► Conversational Agents



M Casadio, T Dinkar, E Komendantskaya, L Arnaboldi, ML Daggitt, O Isac, ... et al. NLP verification: Towards a general methodology for certifying robustness. European Journal of Applied Mathematics. 2025

► Drones



C Kessler, E Komendantskaya, M Casadio, IM Viola, T Flinkow, ... et al. Neural Network Verification for Gliding Drone Control: A Case Study. Symposium on AI Verification: SAIV 2025

► Medical devices



A Sirman, F Conway, J Ciupa, GG Grinbergs, E Komendantskaya, ... et al. Vancomycert: A certified neuro-symbolic drug delivery system (case study) International Symposium on AI Verification: SAIV 2026